Predicting Credit Card Application Status

Nic Kane

Math 448: Professor He

Goal/Motivation

- Building Credit is one of the most important factors in modern economic success.

- A credit card is one of the easiest ways to start building your creditworthiness and credit score.

- I would like to know which factors affect the applicants acceptance status. Which are more important, and which don’t matter.

Project Overview

01
Data Summary

02
Cleaning

03
Exploratory Data Analysis

04
Modeling

05
Model Comparison

06
Conclusion/Next Steps

Data Summary

- Data contains 12 variables. Card is the response variable, and there are 11 predictors.
- From *Econometric Analysis*, 5th edition. (Greene)
- Since I am predicting the status of acceptance I will be using classification methods.
- How clean is the data?

Variable	Description
card	Factor. Was the application for a credit card accepted?
reports	Number of major derogatory reports.
age	Age in years plus twelfths of a year.
income	Yearly income (in USD 10,000).
share	Ratio of monthly credit card expenditure to yearly income.
expenditure	Average monthly credit card expenditure.
owner	Factor. Does the individual own their home?
selfemp	Factor. Is the individual self-employed?
dependents	Number of dependents.
months	Months living at current address.
majorcards	Number of major credit cards held.
active	Number of active credit accounts.

Cleaning

- Turns out that there were a few ages that were less than 1. Removed

```
      age
 Min.   : 0.1667
 1st Qu.:25.4167
 Median :31.2500
 Mean   :33.2131
 3rd Qu.:39.4167
 Max.   :83.5000
```

- Converted the “yes” “no” variables to 1 and 0 for analytical purposes

```
#changing the categorical variables to 1 or 0
credit_clean <- credit_clean %>%
  mutate(
    card = ifelse(card == "yes", 1, 0),
    owner = ifelse(owner == "yes", 1, 0),
    selfemp = ifelse(selfemp == "yes", 1, 0)
  )
```

Exploratory Data Analysis

- First Step: Observe the class balance.

- Weighs heavier on 1 (accepted)

- Solution: Undersampling to get a ~50/50 split so one class does not affect the model.

Before Sampling

0	1
0.2248476	0.7751524

After Sampling

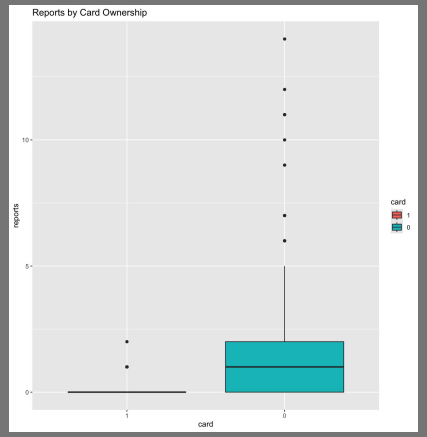
1	0
0.4878472	0.5121528

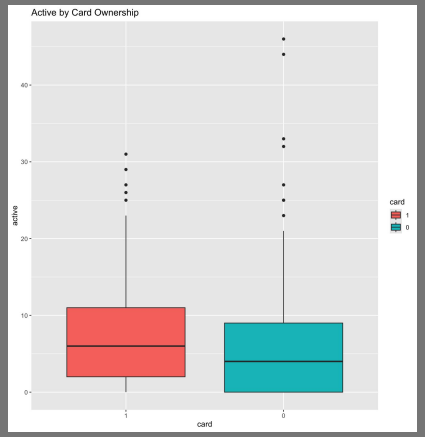
Exploratory Data Analysis

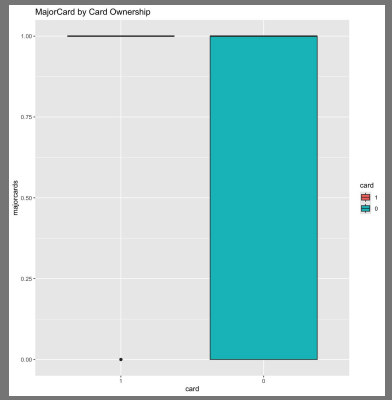
- Next Step: Plot each predictor with the response variable

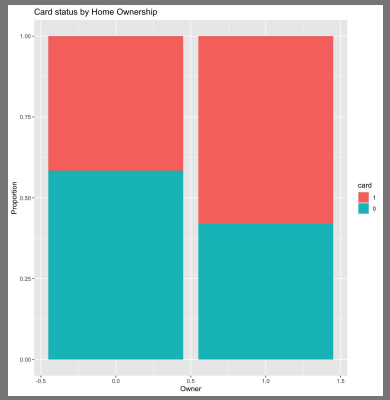
- Predictors that seem significant:

  Reports,
  Active,
  Income,
  Owner



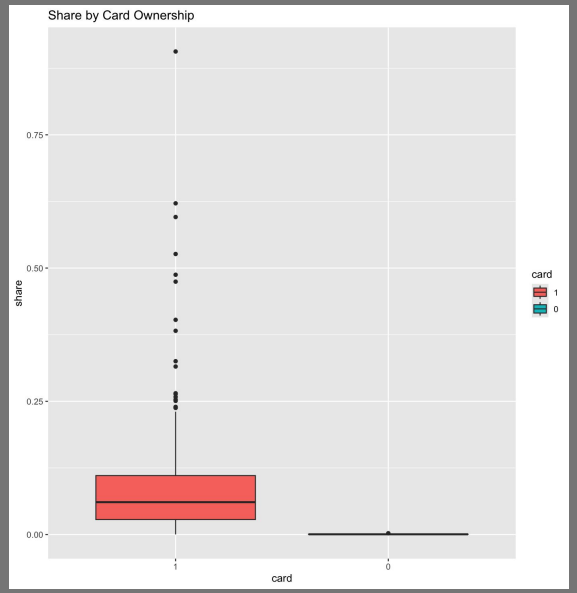


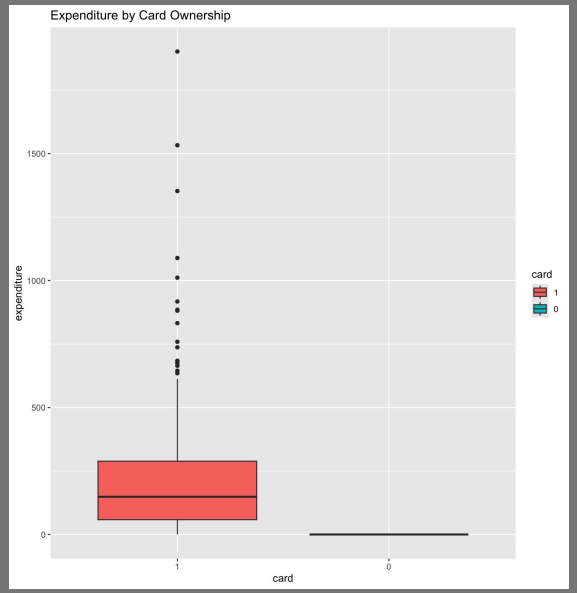




Exploratory Data Analysis

- Expenditure and share have a similar problem: leakage. Both ~0 for those not accepted





Modeling: Test/Train Split

- Split data 80/20 into training and test, to calculate accuracy for each model

- Used set.seed(1) to ensure consistent results

```
# --- create training sets ---
train_index <- sample(seq_len(nrow(credit_modeling)), size = 0.8 * nrow(credit_modeling))

train <- credit_modeling[train_index,]
test <- credit_modeling[-train_index,]
```

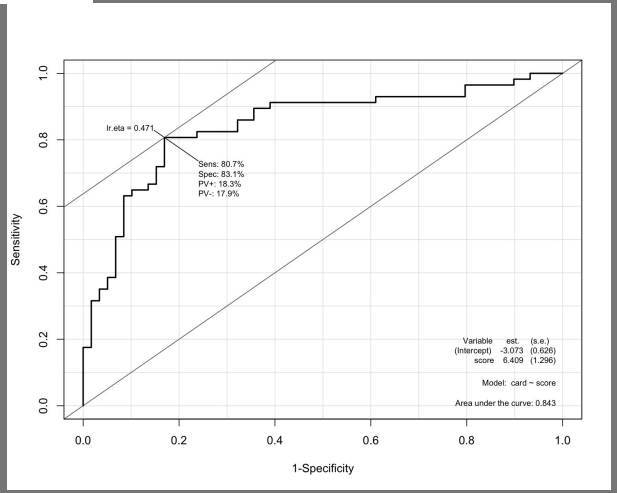
Modeling: Logistic Regression

- Original fit drops owner. Backwards step confirms.
- Confusion matrix is not promising
- Accuracy is not great! (33+5)/(33+5+26+52)=0.33
- AUC = 0.843 !

```
Coefficients:
            Estimate Std. Error z value Pr(>|z|)
(Intercept)  0.77774    0.25914   3.001  0.00269 **
reports      1.45155    0.19213   7.555 4.19e-14 ***
income      -0.25014    0.07643  -3.273  0.00106 **
owner       -0.02906    0.26191  -0.111  0.91165
active      -0.10288    0.02273  -4.526 6.01e-06 ***
---
Signif. codes:  0 ‘***’ 0.001 ‘**’ 0.01 ‘*’ 0.05 ‘.’ 0.1 ‘ ’ 1

(Dispersion parameter for binomial family taken to be 1)

    Null deviance: 637.14  on 459  degrees of freedom
Residual deviance: 475.81  on 455  degrees of freedom
AIC: 485.81

Number of Fisher Scoring iterations: 6
```

```
card ~ reports + income + active

          Df Deviance    AIC
<none>         475.82 483.82
- income   1   489.46 495.46
- active   1   504.60 510.60
- reports  1   617.63 623.63
```

log.pred	1	0
0	26	5
1	33	52

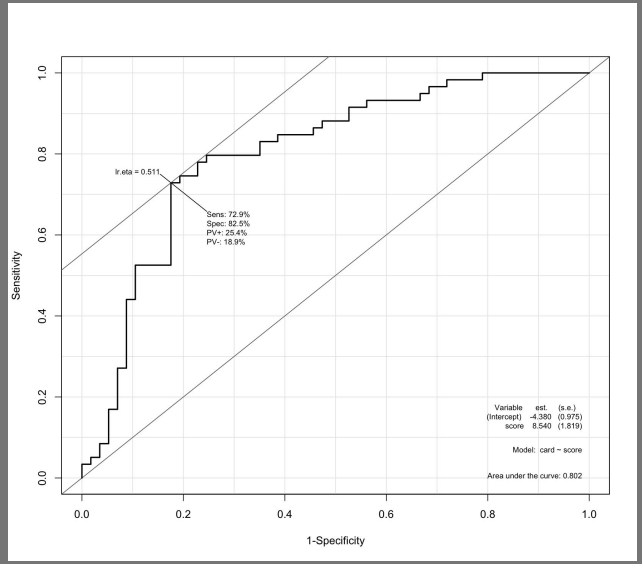


Modeling: LDA

- Also fits owner
- Has much better accuracy than logistic: 0.767
- AUC is smaller, but comparable: 0.802

```
Call:
lda(card ~ reports + income + active, data = train)

Prior probabilities of groups:
        0         1
0.5173913 0.4826087

Group means:
    reports   income   active
0 1.6008403 2.958529 6.134454
1 0.1666667 3.588874 7.644144

Coefficients of linear discriminants:
                LD1
reports -0.54949557
income   0.20795821
active   0.06995476
```

	Actual	
Predicted	0	1
0	43	13
1	14	46

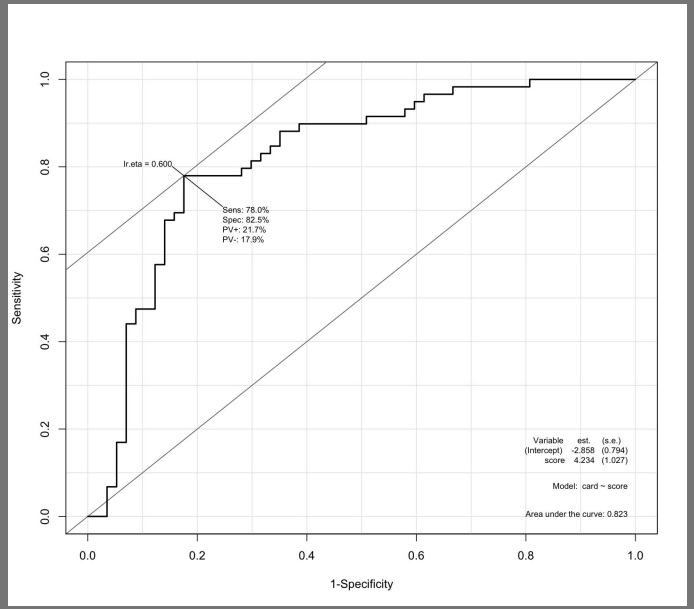


Modeling: QDA

- Accuracy lower than LDA: 0.672
- AUC highest so far: 0.823
- Since more complex than LDA, should favor that

```
Call:
qda(card ~ reports + income + owner + active, data = train)

Prior probabilities of groups:
        0         1
0.5173913 0.4826087

Group means:
    reports   income     owner   active
0 1.6008403 2.958529 0.3151261 6.134454
1 0.1666667 3.588874 0.4909910 7.644144
```

	Actual	
Predicted	0	1
0	23	4
1	34	55



Modeling: KNN

- Tried K=1, 3, 5
- K=3 clearly the best with highest accuracy so far
- AUC for K=3: 0.835
- So far, best balance between accuracy/AUC

```
         test.card
knn.pred  0  1
       0 39 16
       1 18 43
```

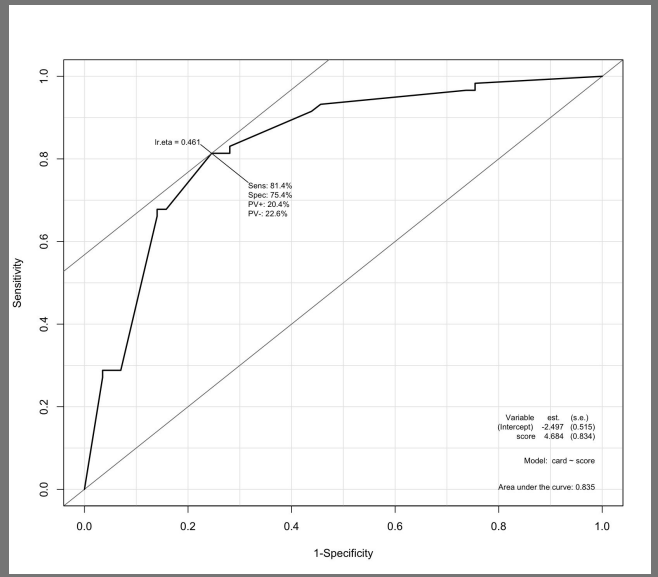
K=1, Acc. = 0.707

```
         test.card
knn.pred  0  1
       0 43  9
       1 14 50
```

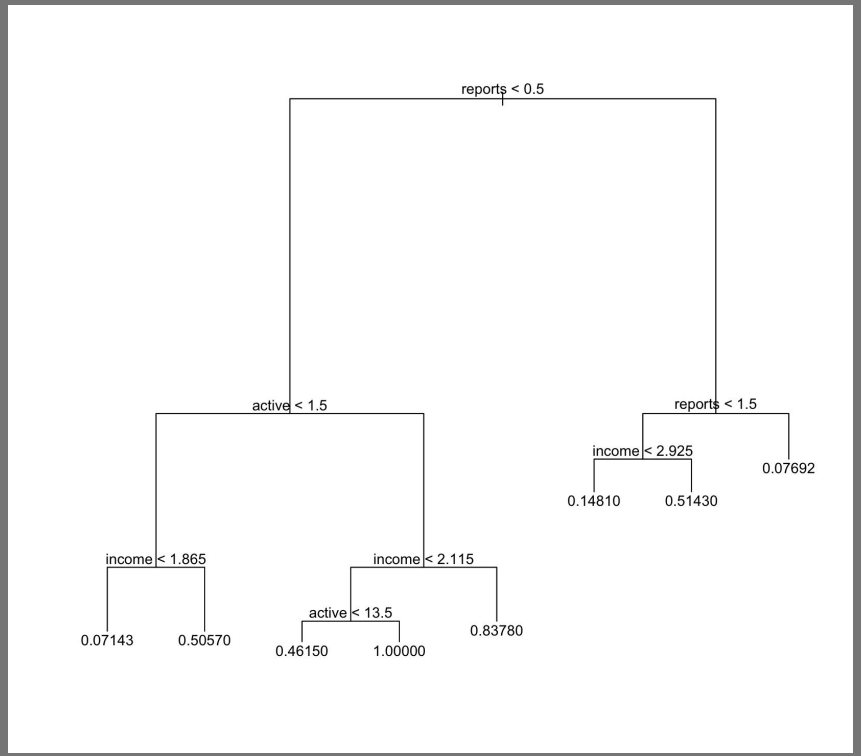
K=3, Acc. = 0.802

```
         test.card
knn.pred  0  1
       0 42 11
       1 15 48
```

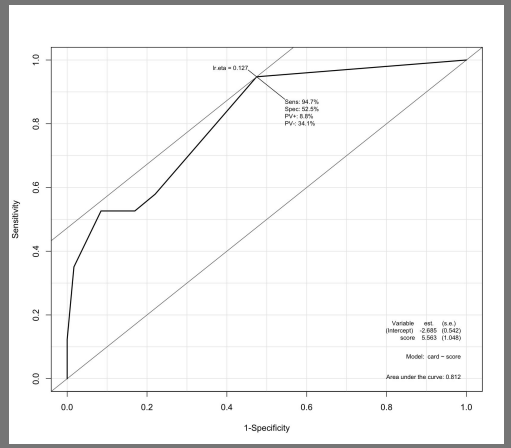
K=5, Acc. = 0.776



Modeling: Classification Trees



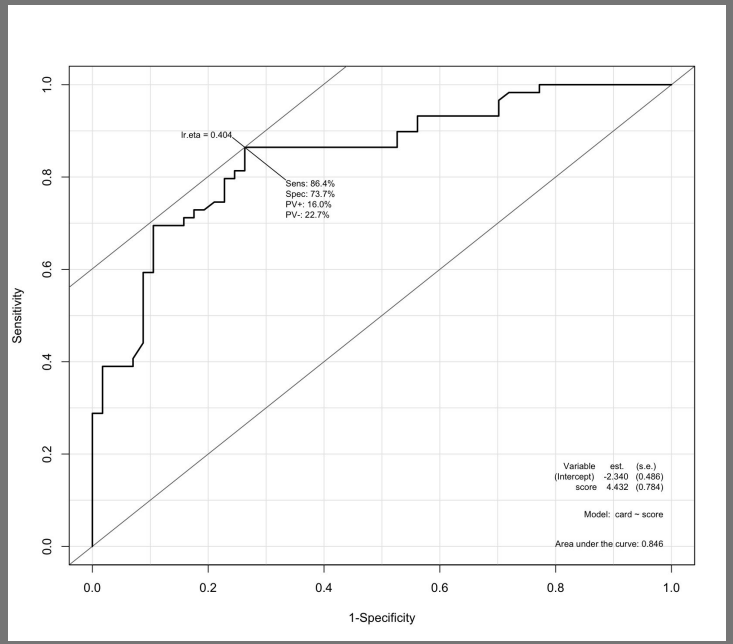
	Actual	
Predicted	1	0
1	46	24
0	13	33



- Accuracy: 0.681
- AUC: 0.812

Note: Pruning did not improve accuracy

Modeling: Random Forests



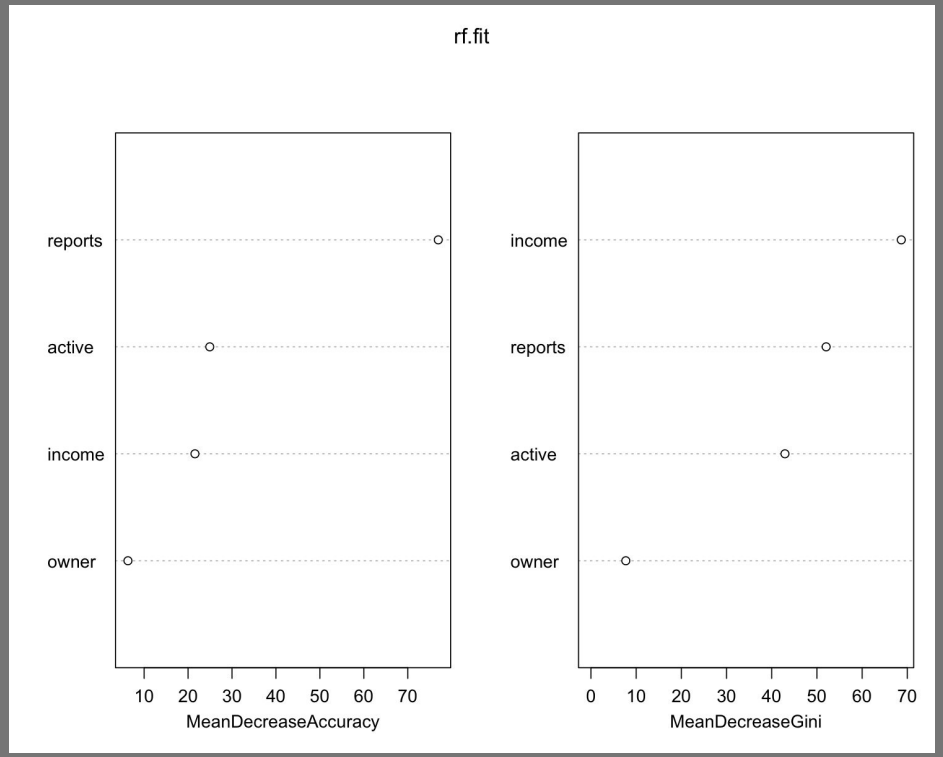

```
Call:
 randomForest(formula = card ~ reports + income + owner + active,      data = train, ntree = 500, mtr
y = 2, importance = TRUE)
               Type of random forest: classification
                     Number of trees: 500
No. of variables tried at each split: 2

        OOB estimate of  error rate: 25.22%
Confusion matrix:
    0   1 class.error
0 171  67   0.2815126
1  49 173   0.2207207
```

- Accuracy: 0.801
- AUC: 0.846

Overall best so far

Modeling: Random Forests (VI Plot)

- Reports/Income are most important by far
- Active has a moderate importance
- Owner is smallest



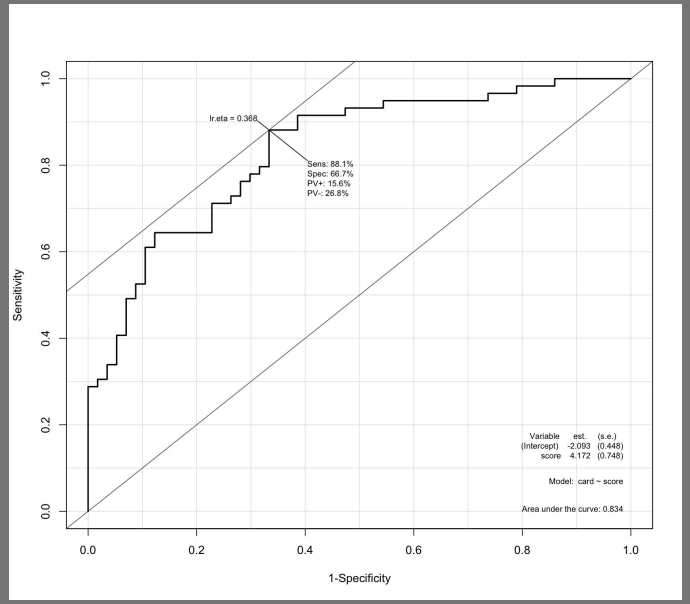
Modeling: Boosting

- Accuracy: 0.715
  Decent, in the middle so far

- AUC: 0.834
  One of the higher values we've seen

```
gbm(formula = card ~ reports + income + owner + active, distribution = "bernoulli",
    data = train, n.trees = 5000, interaction.depth = 3, n.minobsinnode = 10,
    shrinkage = 0.01, bag.fraction = 0.5, verbose = FALSE)
A gradient boosted model with bernoulli loss function.
5000 iterations were performed.
There were 4 predictors of which 4 had non-zero influence.
```

	Actual	
Predicted	0	1
0	41	17
1	16	42



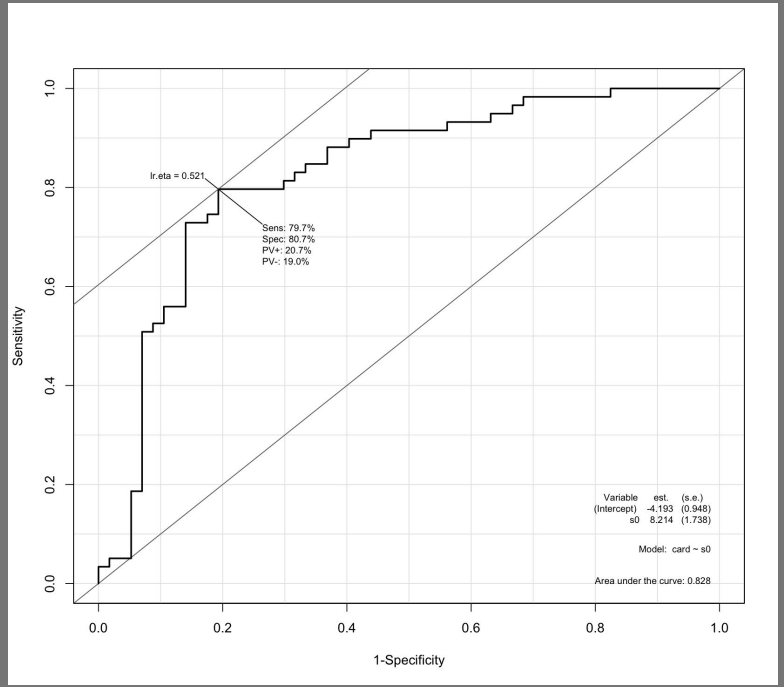
Modeling: Ridge

- Accuracy: 0.759
  Close to the best

- AUC: 0.828
  Close, but not high enough

```
Call:  glmnet(x = X.train, y = y.train, family = "binomial", alpha = 0,      lambda = lambda.ridge)

  Df  %Dev  Lambda
1  4 22.97 0.01896
```

	Actual	
Predicted	0	1
0	39	10
1	18	49



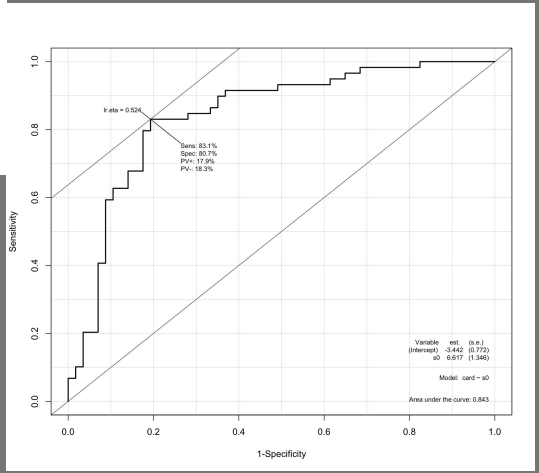
Modeling: Lasso

- Accuracy: 0.759
  This is the same as ridge.

- AUC: 0.843
  Since same accuracy as ridge, we can observe lasso is better.

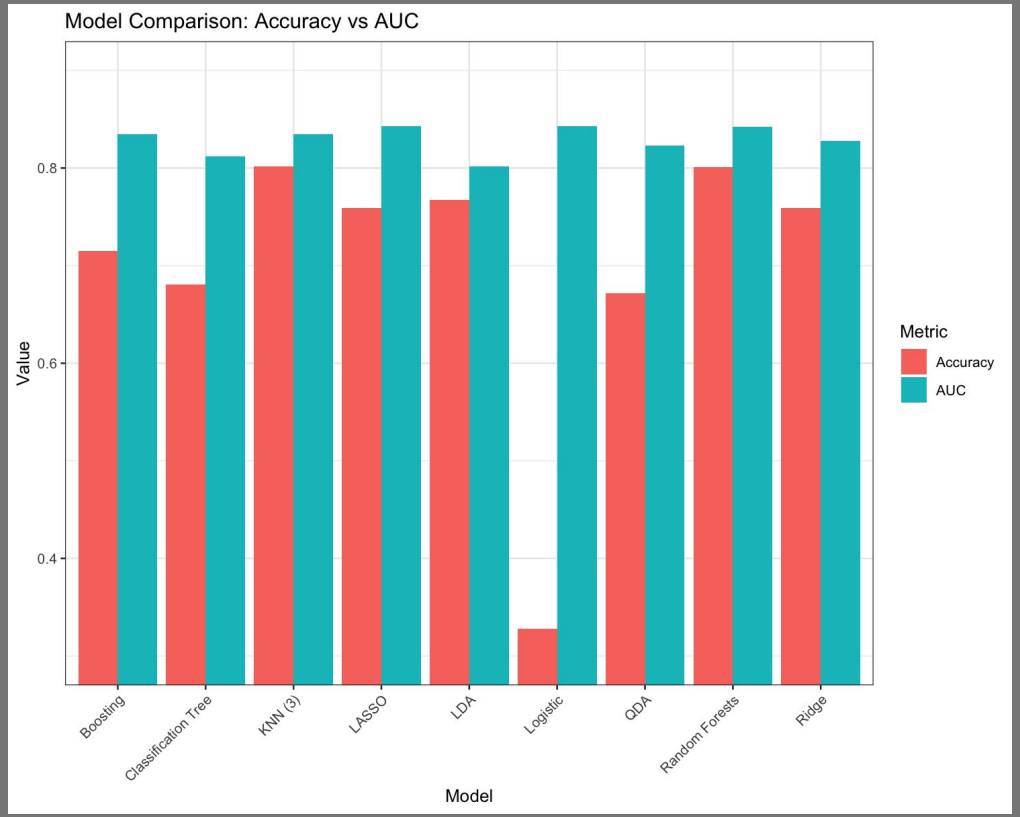
```
Call:  glmnet(x = X.train, y = y.train, family = "binomial", alpha = 1,      lambda = lambda.lasso)

  Df  %Dev   Lambda
1  4 25.29 0.002181
```

	Actual	
Predicted	0	1
0	38	9
1	19	50



Comparison

Model	Accuracy	AUC
Logistic	0.328	**0.843**
LDA	0.767	0.802
QDA	0.672	0.823
KNN (3)	**0.802**	0.835
Classification Tree	0.681	0.812
Random Forests	**0.801**	**0.842**
Boosting	0.715	0.835
Ridge	0.759	0.828
LASSO	0.759	0.843



Conclusion/Next Steps

- Best accuracy was KNN (K=3) and Random Forests.
- Best AUC was logistic and Random Forests.
- Best model for data is random forests as it strikes a good balance between accuracy/AUC.
- Some further considerations
  - Find a larger/more modern data set.
  - Try a different sampling seed for train/test set and compare